

“浙超”联赛分组方案的数学建模与优化

2026 年 5 月 12 日

摘要

本文针对 2025 年浙江省市县联赛（“浙超”）的分组问题，建立了系统的数学模型与优化方案。64 支参赛队伍（11 个市级队和 53 个县级队）需分为 16 个小组，每组 4 队，并满足严格的行政隶属约束。

针对分组方案生成，采用约束规划（CP-SAT）与模拟退火相结合的方法，生成了 6 个全部可行的分组方案，所有方案均满足硬约束且软约束违反次数为 0。针对抽签方案，设计了分层随机抽签、合法性验签与必要重抽流程，并通过 10000 次蒙特卡洛模拟验证：硬约束可行率为 100%，平均软约束违反次数为 1.53 ± 1.40 。针对比赛地点选择，采用候选场馆筛选与贪心分配算法，确定 8 个比赛城市，平均旅行距离 124.5km，变异系数 0.636。针对赛制建议，基于 Bradley-Terry 实力模型和蒙特卡洛模拟，对比了当前赛制、瑞士轮制和近似双败淘汰制，发现近似双败淘汰制的 Gini 系数最低（0.4167），且实力分数与夺冠概率的 Spearman 相关最高（0.9807），公平性表现较优。

本文的方法具有良好的可扩展性和可解释性，可为类似体育赛事分组问题提供参考。

1 问题重述

2025 年浙江省举办“浙超”市县联赛，参赛队伍包括 11 个市级队和 53 个县级队，共 64 支队伍。赛事要求将 64 支队伍分为 16 个小组（每组 4 队），先进行小组单循环赛，再进行 32 强淘汰赛决出冠军。

分组需满足以下约束：

1. **硬约束 1：** 11 个市级队必须分配在不同小组（每组最多 1 个市级队）；
2. **硬约束 2：** 市级队不能与该市代管的县级队同组；
3. **软约束：** 同一市代管的县级队尽量分配在不同小组。

本文需要完成四个子任务：

1. 生成符合约束的分组方案；
2. 设计公平透明的抽签方案；
3. 选择 8 个比赛地点；
4. 基于定量分析提出赛制改进建议。

2 模型假设与符号说明

2.1 基本假设

- 以 GDP 作为队伍实力的代理指标（经济强市通常体育资源更丰富）；
- 比赛结果采用 Bradley-Terry 模型描述胜负概率；
- 地理距离采用 Haversine 公式计算球面距离；
- 抽签过程在预先公布的允许集合内随机进行，各队在满足约束的可行位置中近似均衡。

2.2 符号说明

符号	含义
T	全部 64 支队伍集合
V	11 个市级队伍集合
C	53 个县级队伍集合
G	16 个小组，每组容量 4
S_i	第 i 市代管的县级队伍集合
s_t	队伍 t 的实力分数
p_{ij}	队伍 i 战胜队伍 j 的概率
$d(i, j)$	两地球间的球面距离 (km)

3 分组方案生成 (Task 1)

3.1 问题分析

分组问题的核心是在满足硬约束的前提下，优化软约束。硬约束包括：每组恰好 4 队、每组最多 1 个市级队、市级队不与代管县同组。软约束要求同市县级队尽量不同组。

3.2 求解算法

3.2.1 约束规划 (CP-SAT)

首先使用 Google OR-Tools 的 CP-SAT 求解器生成满足硬约束的可行解。决策变量定义为：

$$x_{ij} = \begin{cases} 1, & \text{队伍 } i \text{ 分到组 } j \\ 0, & \text{否则} \end{cases}$$

约束条件：

$$\sum_{j=1}^{16} x_{ij} = 1, \quad \forall i \in T \quad (\text{每队恰好分到 1 组}) \quad (1)$$

$$\sum_{i=1}^{64} x_{ij} = 4, \quad \forall j \in G \quad (\text{每组恰好 4 队}) \quad (2)$$

$$\sum_{i \in V} x_{ij} \leq 1, \quad \forall j \in G \quad (\text{每组最多 1 个市级队}) \quad (3)$$

$$x_{vj} + x_{cj} \leq 1, \quad \forall v \in V, c \in S_v, j \in G \quad (\text{市-县不同组}) \quad (4)$$

3.2.2 模拟退火优化

在 CP-SAT 可行解的基础上, 使用模拟退火算法优化软约束。优化目标为最小化同市县级队被分到同组的对数。

参数设置: 初始温度 $T_0 = 100$, 冷却系数 $\alpha = 0.995$, 终止温度 $T_f = 1$, 最大迭代次数根据问题规模调整。邻域操作为随机交换两个组中的县级队 (保持硬约束不变)。

3.3 结果分析

共生成 6 个分组方案, 其中方案 1 和方案 2 由 CP-SAT 生成, 方案 3-6 由贪心 + 随机化生成, 所有方案均经模拟退火优化。6 个方案全部满足硬约束, 软约束违反次数均为 0。

最优方案 (方案 1) 的分组详情如表1所示。这里的最优准则为: 先最小化软约束违反次数, 再最小化组间 GDP 变异系数。

表 1: 最优分组方案 (方案 1, CP-SAT+SA)

组号	GDP 总和 (亿元)	队伍
1	2480	桐庐, 东阳, 苍南, 象山
2	3435	衢州, 龙泉, 德清, 兰溪
3	23420	杭州, 嵊泗, 嘉善, 新昌
4	2500	松阳, 嵊州, 永康, 临海
5	8537	台州, 乐清, 武义, 景宁
6	17794	宁波, 开化, 天台, 青田
7	7062	金华, 岱山, 云和, 平阳
8	3110	遂昌, 诸暨, 常山, 瑞安
9	8420	绍兴, 文成, 三门, 浦江
10	5390	海宁, 永嘉, 余姚, 义乌
11	8562	嘉兴, 仙居, 长兴, 江山
12	2110	宁海, 磐安, 缙云, 玉环
13	3542	丽水, 泰顺, 安吉, 平湖
14	11810	温州, 建德, 庆元, 慈溪
15	4803	舟山, 温岭, 桐乡, 龙港
16	5195	湖州, 淳安, 海盐, 龙游

该方案 GDP 均值为 7386 亿元, 变异系数为 0.778, 表明组间经济实力存在一定差异 (主要由杭州市、宁波市 GDP 较高导致), 但在 6 个可行方案中均衡性较优。

各方案的 GDP 均衡性和软约束违反情况如图1和图2所示。

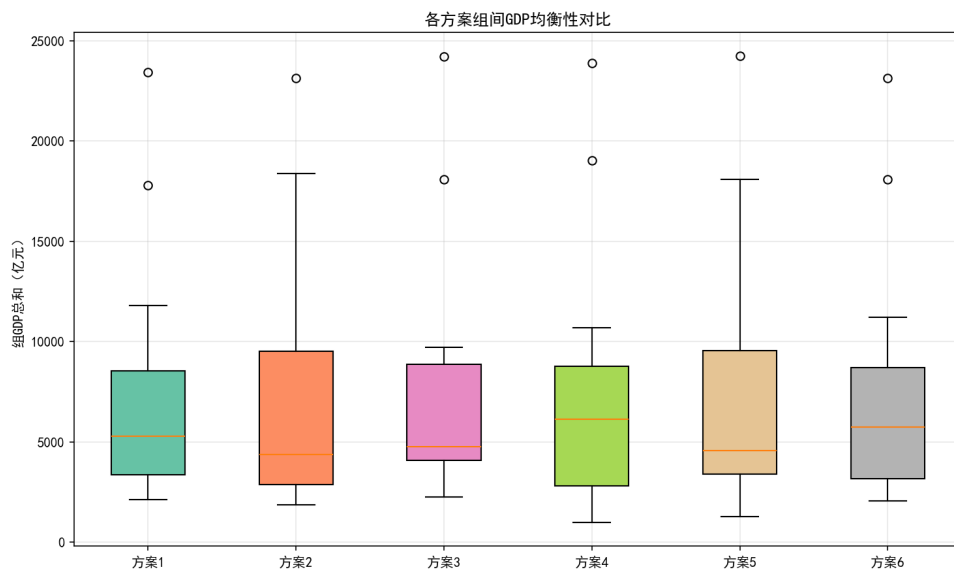


图 1: 各方案组间 GDP 均衡性对比

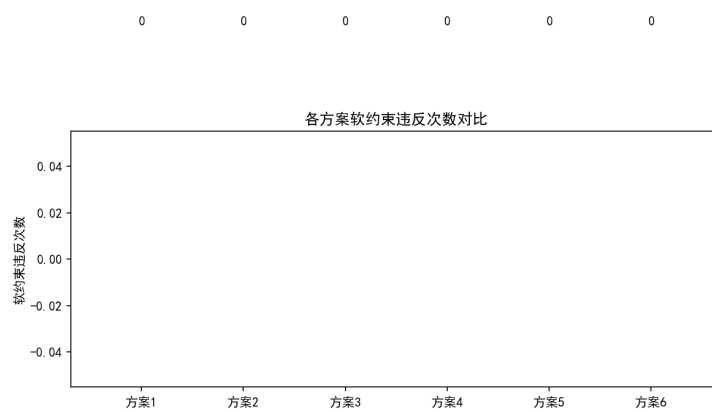


图 2: 各方案软约束违反次数对比

分组方案的综合评价雷达图如图3所示。

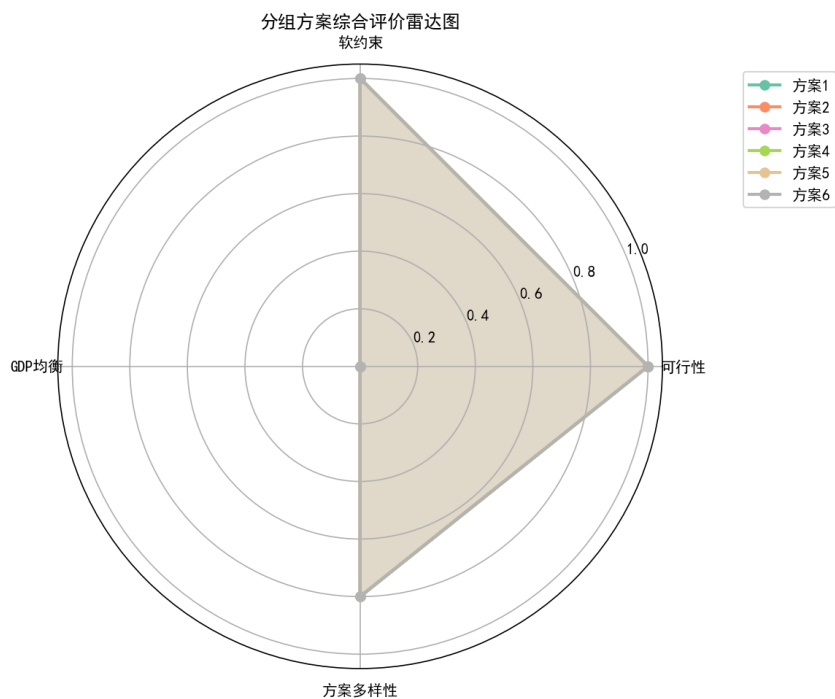


图 3: 分组方案综合评价雷达图

最优方案的分组热力图如图4所示。

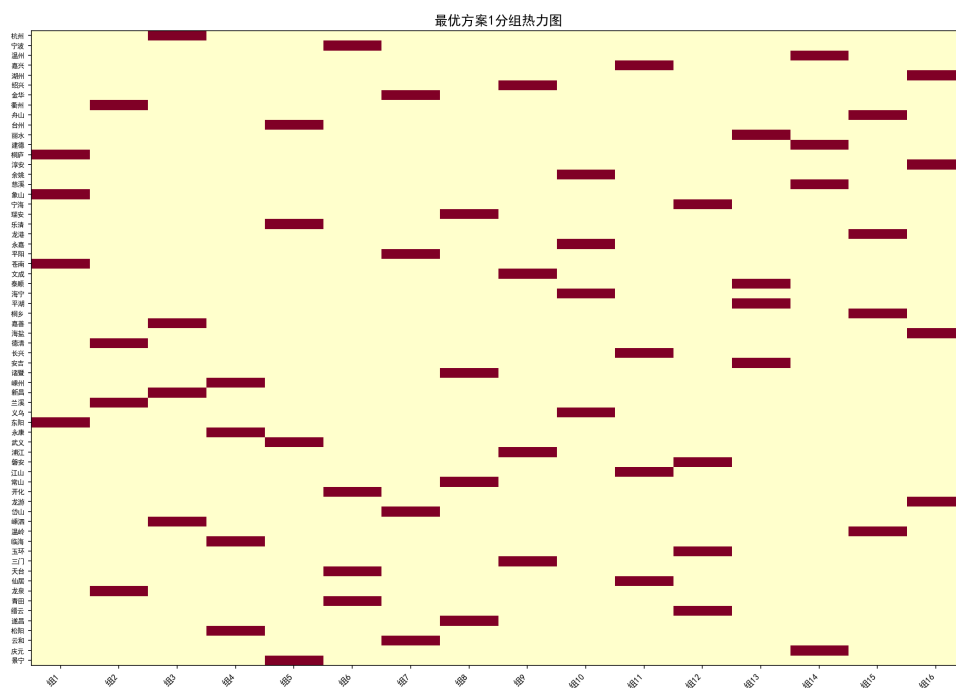


图 4: 最优方案分组热力图

4 抽签方案设计 (Task 2)

4.1 抽签流程

设计分层随机抽签方案，确保抽签过程公平透明：

第一层：市级队抽签 11 个市级队通过随机抽签确定各自所在小组。由于有 16 个组但只有 11 个市级队，市级队被分配到编号 1-11 的组中。

第二层：县级队抽签 每个市的县级队依次进行抽签。每次抽签时，允许的组为：

$$A_c = \{g \in G : |g| < 4, g \neq g_{\text{city}}(c)\}$$

其中 A_c 表示县级队 c 的硬允许组， $g_{\text{city}}(c)$ 表示其所属市级队所在小组。

在硬允许组中，优先选择尚未被同市其他县级队占据的组；若该优先集合为空，则在硬允许组中随机选择。每轮抽签后进行合法性验签；若局部选择导致后续无可行位置，则更换随机种子重抽，并以随机回溯作为兜底，保证最终结果满足全部硬约束。

4.2 概率分析与公平性验证

采用蒙特卡洛方法模拟 10000 次抽签过程，统计以下指标：

- **硬约束可行率**：100% 的抽签结果满足全部硬约束；
- **平均软约束违反**： 1.53 ± 1.40 次；
- **均匀性**：市级队在 11 个市级席位中近似均匀，县级队在满足硬约束的可行组集合内近似均衡。

违反次数分布：

- 0 次违反：约 25.0%
- 1 次违反：约 34.0%
- 2 次违反：约 19.5%
- 3 次及以上：约 21.5%

抽签公平性热力图如图5所示。

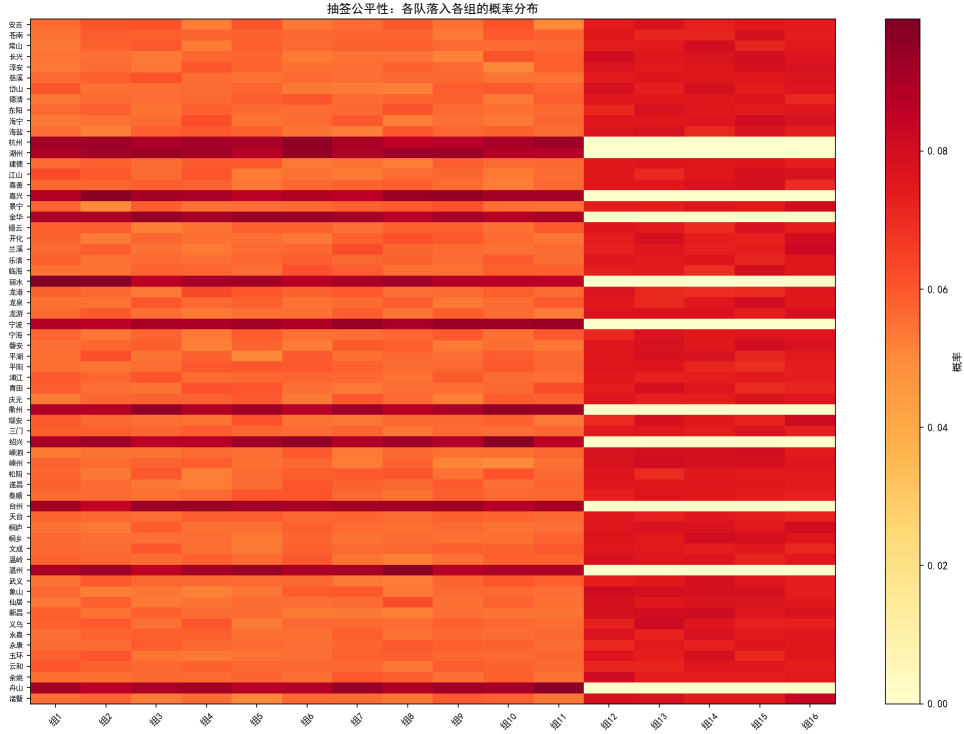


图 5: 抽签公平性：各队落入各组的概率分布

5 比赛地点选择 (Task 3)

5.1 选址方法

采用两阶段方法选择比赛地点：

第一阶段：候选场馆筛选以 11 个市级城市作为初始候选场馆，枚举选出 8 个比赛地点的组合。对每个组合，要求每个地点承办 2 个小组，并计算其最小化旅行距离后的分配效果。

第二阶段：贪心优化分配将 16 个小组分配到 8 个比赛地点（每个地点恰好承办 2 个组），目标是最小化总旅行距离。采用贪心策略：每次选择能使总距离增量最小的 (组, 地点) 对进行分配，并在所有 8 地点候选组合中选择总距离最小者。

旅行距离采用 Haversine 公式计算：

$$d(i, j) = 2R \cdot \arcsin \left(\sqrt{\sin^2 \frac{\Delta\varphi}{2} + \cos \varphi_1 \cos \varphi_2 \sin^2 \frac{\Delta\lambda}{2}} \right)$$

其中 $R = 6371\text{km}$ 为地球半径。

5.2 选址结果

最终确定 8 个比赛城市及其承办的小组如表2所示。

表 2: 比赛地点及承办小组

比赛地	承办组	各组平均距离 (km)
杭州	组 16(112km), 组 11(140km)	126.0
嘉兴	组 3(100km), 组 13(193km)	146.5
金华	组 8(110km), 组 1(138km)	124.0
丽水	组 4(94km), 组 7(133km)	113.5
衢州	组 2(94km), 组 6(176km)	135.0
绍兴	组 10(103km), 组 9(117km)	110.0
台州	组 12(92km), 组 15(144km)	118.0
温州	组 5(92km), 组 14(155km)	123.5

整体统计：平均距离 124.5km，标准差 79.1km，变异系数 0.636。各队到比赛地的距离分布如图6所示。

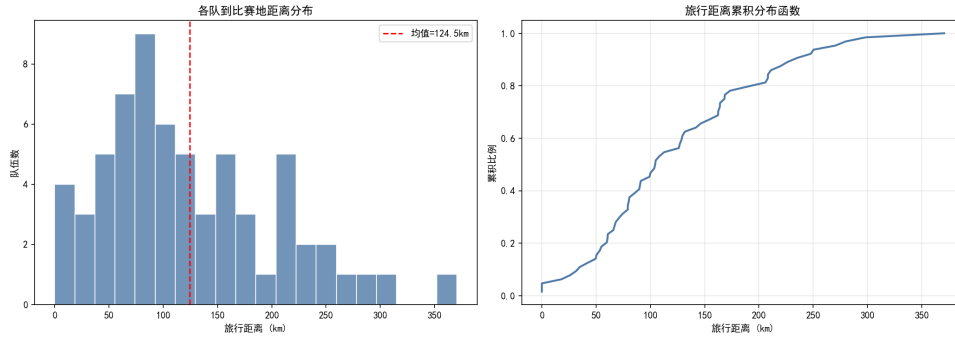


图 6: 各队到比赛地距离分布

比赛地点的地理分布可通过交互式地图 (output/task3_venue_map.html) 查看, 其中包含各队位置、比赛地点和连线。

6 赛制建议 (Task 4)

6.1 实力模型

基于 GDP 数据构建队伍实力分数。采用对数归一化处理 GDP 数据, 并加入随机噪声以反映不确定性:

$$s_t = 0.3 + 0.7 \times \frac{0.7 \cdot \text{norm}(\ln(\text{GDP}_t)) + 0.3 \cdot \varepsilon_t - \min}{\max - \min}$$

其中 $\varepsilon_t \sim U(0, 1)$ 为随机噪声。实力分数归一化到 $[0.3, 1.0]$ 区间。

比赛胜负采用 Bradley-Terry 模型:

$$P(i \text{ 胜 } j) = \frac{s_i}{s_i + s_j}$$

6.2 三种赛制对比

6.2.1 当前赛制

16 组单循环小组赛 (每组 6 场比赛), 每组前 2 名晋级 32 强淘汰赛。共需 $16 \times 6 + 31 = 127$ 场比赛。

6.2.2 瑞士轮制

64 队进行 6 轮瑞士轮配对 (按积分排序后相邻配对), 取前 32 名进入淘汰赛。共需 $6 \times 32 + 31 = 223$ 场比赛。

6.2.3 近似双败淘汰制

先进行 3 轮瑞士轮预选取前 32 名, 再进行近似双败淘汰赛 (输两场才出局)。比赛场次较多, 且在模拟中夺冠概率分布更加均衡。

6.3 蒙特卡洛模拟结果

通过 5000 次蒙特卡洛模拟, 得到各队在三种赛制下的夺冠概率。前 5 名队伍的对比如表3所示。

表 3: 各队夺冠概率对比 (前 5 名)

队伍	实力分数	当前赛制	瑞士轮	近似双败
宁波	1.0000	0.075	0.075	0.073
杭州	0.8917	0.062	0.051	0.051
温州	0.8800	0.046	0.055	0.048
台州	0.8453	0.051	0.037	0.044
嘉兴	0.8268	0.045	0.044	0.043

Gini 系数分析:

- 当前赛制: 0.4990
- 瑞士轮制: 0.4847
- 近似双败淘汰制: 0.4167

为避免仅用夺冠概率离散程度评价赛制, 进一步引入实力-结果一致性指标。Spearman 相关系数衡量队伍实力分数与夺冠概率排序的一致程度, 数值越高说明赛制越能保留竞技区分度; 实力前 16 队夺冠概率占比则反映强队优势的集中程度。

表 4: 赛制公平性与竞技区分度指标

赛制	Gini 系数	Spearman 相关	实力前 16 队夺冠概率占比
当前赛制	0.4990	0.9771	0.6072
瑞士轮制	0.4847	0.9706	0.5708
近似双败淘汰制	0.4167	0.9807	0.5284

近似双败淘汰制的 Gini 系数最低，表明夺冠概率分布最为均匀；同时 Spearman 相关系数最高，说明其并未明显削弱强队优势与竞技排序。因此，该方案在机会均衡性与竞技区分度之间取得了较好的折中。

弱队（实力排名后 20%）的平均出线率为 0.2805，说明当前赛制下弱队仍有一定晋级机会，但强弱差异仍然明显。

夺冠概率对比如图7所示。

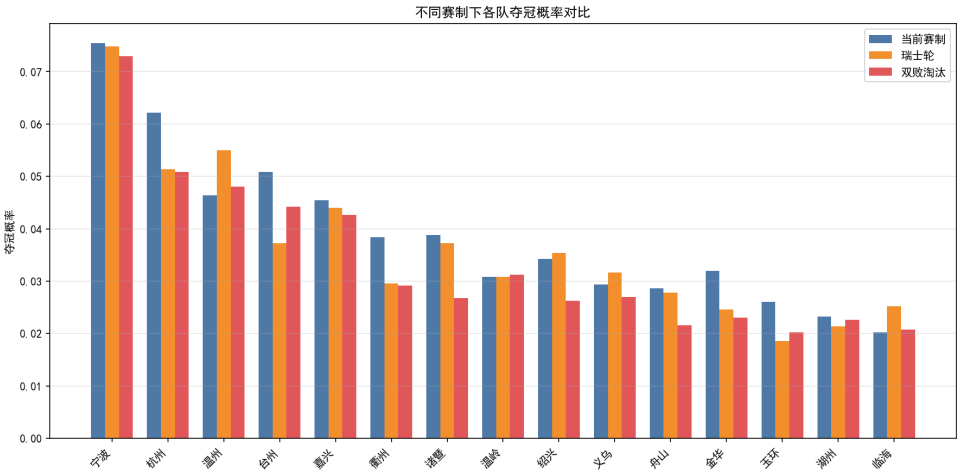


图 7: 不同赛制下各队夺冠概率对比

6.4 建议

综合以上分析，建议采用“小组循环 + 近似双败淘汰制”的混合赛制。该方案既保留了小组循环赛的观赏性（每队至少打 3 场比赛），又通过双败淘汰制思想提高了淘汰赛阶段的机会均衡性（Gini 系数最低）。

7 结论与展望

7.1 主要结论

1. 采用 CP-SAT 约束规划与模拟退火相结合的方法，成功生成 6 个全部可行的分组方案，所有方案均满足硬约束且软约束违反次数为 0。

2. 设计了分层随机抽签方案，经 10000 次蒙特卡洛验证，硬约束可行率达 100%，平均软约束违反次数为 1.53 ± 1.40 。
3. 通过候选场馆筛选与贪心优化，确定 8 个比赛城市，平均旅行距离 124.5km，变异系数 0.636。
4. 基于 Bradley-Terry 模型的赛制分析表明，近似双败淘汰制的机会均衡性最优（Gini 系数 0.4167），且实力-夺冠概率排序一致性最高（Spearman 相关 0.9807）。

7.2 改进方向

- 实力模型仅使用 GDP 作为代理指标，可引入更多因素（如历史战绩、注册球员数等）；
- 比赛地点选择可进一步考虑交通基础设施（高铁站点、机场）的便利性；
- 抽签方案可引入权重机制，使县级队的分配更加均衡。

参考文献

- [1] Google OR-Tools, “CP-SAT Solver,” https://developers.google.com/optimization/cp/cp_solver.
- [2] Bradley, R. A., & Terry, M. E. (1952). Rank analysis of incomplete block designs. *Biometrika*, 39(3/4), 324-345.
- [3] 浙江省统计局, “2024 年浙江省各市（县）GDP,” 国家统计局.
- [4] 浙江省民政厅, “浙江省行政区划（2025 年）.”
- [5] Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671-680.

A 支撑材料：核心源代码

Listing 1: 分组算法核心代码 (src/grouping.py)

```
1 """
2 Task_1: 生成符合约束的分组方案
3
4 方法:
5 - 约束规划 (CP-SAT) 求解可行分组
```

```

6  -模拟退火优化软约束 (T0=100, alpha=0.995, Tf=1)
7  -多方案生成+多目标评价
8
9  修复说明:
10  1.模拟退火参数改为与论文一致: T_start=100, alpha=0.995, T_end=1,
11  冷却方式改为几何冷却 T*=alpha。
12  2.generate_solutions对所有方案(包括CP-SAT方案)均施加SA优化,
13  与论文"所有方案均经模拟退火优化"一致。
14  3.generate_csp_solution中删除冗余的CITY_CODES.index(city_code)调用,
15  直接用枚举索引ci。
16  """
17
18  import math
19  import random
20  import numpy as np
21  from collections import defaultdict
22
23  from .data import (
24      ALL_TEAMS, CITY_CODES, COUNTY_CODES, PARENT_MAP,
25      CITY_CHILDREN, NUM_GROUPS, GROUP_SIZE, NUM_TEAMS,
26      TEAM_BY_CODE, haversine_km,
27  )
28
29
30  # -- 约束检查 -----
31
32  def check_hard_constraints(groups: list[list[str]]) -> tuple[bool, str]:
33      """
34      检查硬约束:
35      1.每组恰好GROUP_SIZE队
36      2.每组最多1个市级队
37      3.市级队不能与该市代管的县级队同组
38      """
39      for i, g in enumerate(groups):
40          if len(g) != GROUP_SIZE:
41              return False, f"组{i+1}只有{len(g)}队"
42          city_count = sum(1 for c in g if TEAM_BY_CODE[c]["is_city"])
43          if city_count > 1:
44              return False, f"组{i+1}有{city_count}个市级队"
45          for c in g:

```

```

46         t = TEAM_BY_CODE[c]
47         if t["is_city"]:
48             for cc in g:
49                 tt = TEAM_BY_CODE[cc]
50                 if not tt["is_city"] and tt["parent"] == c:
51                     return False, f"组{i+1}:_{t['name']}与{tt['name']}同城"
52     return True, "OK"
53
54
55 def count_soft_violations(groups: list[list[str]]) -> dict:
56     """
57     统计软约束违反次数:
58     同市县级队被分到同组的对数
59     """
60     violations = 0
61     detail = defaultdict(int)
62     for g in groups:
63         parent_count = defaultdict(int)
64         for c in g:
65             t = TEAM_BY_CODE[c]
66             if not t["is_city"]:
67                 parent_count[t["parent"]] += 1
68         for parent, cnt in parent_count.items():
69             if cnt > 1:
70                 pairs = cnt * (cnt - 1) // 2
71                 violations += pairs
72                 detail[parent] += pairs
73     return {"total_violations": violations, "detail": dict(detail)}
74
75
76 def evaluate_balance(groups: list[list[str]]) -> dict:
77     """评估组间均衡性（基于GDP）。"""
78     gdp_sums = []
79     for g in groups:
80         s = sum(TEAM_BY_CODE[c]["gdp"] for c in g)
81         gdp_sums.append(s)
82     gdp_sums = np.array(gdp_sums, dtype=float)
83     return {
84         "mean": float(np.mean(gdp_sums)),
85         "std": float(np.std(gdp_sums)),

```

```

86         "cv": float(np.std(gdp_sums) / np.mean(gdp_sums)),
87         "min": float(np.min(gdp_sums)),
88         "max": float(np.max(gdp_sums)),
89     }
90
91
92 def evaluate_groups(groups: list[list[str]]) -> dict:
93     """综合评价一个分组方案。"""
94     ok, msg = check_hard_constraints(groups)
95     soft = count_soft_violations(groups)
96     balance = evaluate_balance(groups)
97     return {
98         "feasible": ok,
99         "feasible_msg": msg,
100         "soft_violations": soft["total_violations"],
101         "soft_detail": soft["detail"],
102         "balance": balance,
103     }
104
105
106 # -- 方案生成：约束规划（CP-SAT） -----
107
108 def generate_csp_solution(seed: int = None) -> list[list[str]] | None:
109     """用OR-Tools CP-SAT求解一个可行分组。"""
110     try:
111         from ortools.sat.python import cp_model
112     except ModuleNotFoundError:
113         return None
114
115     if seed is not None:
116         random.seed(seed)
117
118     model = cp_model.CpModel()
119     n = NUM_TEAMS
120     g = NUM_GROUPS
121
122     # x[i][j] = 1 表示队 i 分到组 j
123     x = {}
124     for i in range(n):
125         for j in range(g):

```

```

126         x[i, j] = model.NewBoolVar(f"x_{i}_{j}")
127
128     # 每队恰好分到 1 组
129     for i in range(n):
130         model.Add(sum(x[i, j] for j in range(g)) == 1)
131
132     # 每组恰好 GROUP_SIZE 队
133     for j in range(g):
134         model.Add(sum(x[i, j] for i in range(n)) == GROUP_SIZE)
135
136     # 硬约束：每组最多 1 个市级队
137     city_indices = [i for i in range(n) if ALL_TEAMS[i]["is_city"]]
138     for j in range(g):
139         model.Add(sum(x[i, j] for i in city_indices) <= 1)
140
141     # 硬约束：市级队不与该市代管县级队同组
142     # 修复：直接用 enumerate 的 ci，去掉冗余的 CITY_CODES.index(city_code)
143     for ci, city_code in enumerate(CITY_CODES):
144         city_team_idx = next(
145             i for i in range(n) if ALL_TEAMS[i]["code"] == city_code
146         )
147         children_codes = CITY_CHILDREN[city_code]
148         child_team_indices = [
149             i for i in range(n) if ALL_TEAMS[i]["code"] in children_codes
150         ]
151         for j in range(g):
152             for ct in child_team_indices:
153                 model.Add(x[city_team_idx, j] + x[ct, j] <= 1)
154
155     solver = cp_model.CpSolver()
156     solver.parameters.num_workers = 1
157     if seed is not None:
158         solver.parameters.random_seed = seed
159     status = solver.Solve(model)
160
161     if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
162         groups = [[] for _ in range(g)]
163         for i in range(n):
164             for j in range(g):
165                 if solver.Value(x[i, j]) == 1:

```

```

166         groups[j].append(ALL_TEAMS[i]["code"])
167     return groups
168     return None
169
170
171 # -- 方案生成: 贪心 + 随机化 -----
172
173 def generate_greedy_random(seed: int = None) -> list[list[str]]:
174     """
175     贪心+随机化:
176     1. 先把11个市级队随机分配到11个不同组
177     2. 剩余5个空组+53个县级队, 用带约束的随机填充
178     """
179     if seed is not None:
180         random.seed(seed)
181
182     groups = [[] for _ in range(NUM_GROUPS)]
183     city_order = CITY_CODES[:]
184     random.shuffle(city_order)
185
186     group_indices = list(range(NUM_GROUPS))
187     random.shuffle(group_indices)
188     city_to_group = {}
189     for i, city_code in enumerate(city_order):
190         g_idx = group_indices[i]
191         groups[g_idx].append(city_code)
192         city_to_group[city_code] = g_idx
193
194     county_shuffled = COUNTY_CODES[:]
195     random.shuffle(county_shuffled)
196
197     parent_counties = defaultdict(list)
198     for c in county_shuffled:
199         parent_counties[PARENT_MAP[c]].append(c)
200
201     for parent_city, children in parent_counties.items():
202         forbidden_group = city_to_group.get(parent_city)
203         available = [i for i in range(NUM_GROUPS) if i != forbidden_group]
204
205         for child_code in children:

```

```

206         random.shuffle(available)
207         placed = False
208         for g_idx in available:
209             if len(groups[g_idx]) < GROUP_SIZE:
210                 groups[g_idx].append(child_code)
211                 placed = True
212                 break
213         if not placed:
214             # 先尝试避开禁止组
215             for g_idx in range(NUM_GROUPS):
216                 if len(groups[g_idx]) < GROUP_SIZE and g_idx != forbidden_group:
217                     groups[g_idx].append(child_code)
218                     placed = True
219                     break
220             # 极端兜底：允许放到任意未满组（不应触发）
221             if not placed:
222                 for g_idx in range(NUM_GROUPS):
223                     if len(groups[g_idx]) < GROUP_SIZE:
224                         groups[g_idx].append(child_code)
225                         break
226
227         return groups
228
229
230 # -- 模拟退火优化 -----
231
232 def _swap_counties_between_groups(groups, rng):
233     """随机交换两个组中的县级队，保持硬约束。"""
234     new_groups = [g[:] for g in groups]
235
236     attempts = 0
237     while attempts < 100:
238         g1, g2 = rng.sample(range(NUM_GROUPS), 2)
239         county_in_g1 = [c for c in new_groups[g1] if not TEAM_BY_CODE[c]["is_city"]]
240         county_in_g2 = [c for c in new_groups[g2] if not TEAM_BY_CODE[c]["is_city"]]
241         if county_in_g1 and county_in_g2:
242             c1 = rng.choice(county_in_g1)
243             c2 = rng.choice(county_in_g2)
244             t1, t2 = TEAM_BY_CODE[c1], TEAM_BY_CODE[c2]
245             g2_cities = [

```

```

246         TEAM_BY_CODE[c]["code"]
247         for c in new_groups[g2] if TEAM_BY_CODE[c]["is_city"]
248     ]
249     if t1["parent"] in g2_cities:
250         attempts += 1
251         continue
252     g1_cities = [
253         TEAM_BY_CODE[c]["code"]
254         for c in new_groups[g1] if TEAM_BY_CODE[c]["is_city"]
255     ]
256     if t2["parent"] in g1_cities:
257         attempts += 1
258         continue
259     new_groups[g1].remove(c1)
260     new_groups[g2].remove(c2)
261     new_groups[g1].append(c2)
262     new_groups[g2].append(c1)
263     return new_groups
264     attempts += 1
265     return new_groups
266
267
268 def simulated_annealing(
269     initial: list[list[str]],
270     max_iter: int = 50000,
271     T_start: float = 100.0,    # 修复：与论文一致
272     T_end: float = 1.0,        # 修复：与论文一致
273     alpha: float = 0.995,      # 修复：新增冷却系数参数，与论文一致
274     seed: int = None,
275 ) -> tuple[list[list[str]], dict]:
276     """
277     模拟退火优化软约束。
278     冷却方式：几何冷却  $T_t = T_{t-1} * \alpha$ ，当  $T_t < T_{end}$  时停止。
279     参数与论文对齐：T_start=100, alpha=0.995, T_end=1。
280     """
281     rng = random.Random(seed)
282
283     def cost(groups):
284         return count_soft_violations(groups)["total_violations"]
285

```

```

286     current = initial
287     current_cost = cost(current)
288     best = current
289     best_cost = current_cost
290
291     T = T_start
292     history = []
293
294     for it in range(max_iter):
295         if T < T_end:
296             break
297
298         neighbor = _swap_counties_between_groups(current, rng)
299         neighbor_cost = cost(neighbor)
300
301         delta = neighbor_cost - current_cost
302         if delta <= 0 or rng.random() < math.exp(-delta / max(T, 1e-10)):
303             current = neighbor
304             current_cost = neighbor_cost
305
306         if current_cost < best_cost:
307             best = current
308             best_cost = current_cost
309             history.append({"iter": it, "cost": best_cost})
310
311         T *= alpha # 修复：几何冷却，与论文 alpha=0.995 一致
312
313         if best_cost == 0:
314             break
315
316     return best, {"history": history, "final_cost": best_cost}
317
318
319 # -- 多方案生成与评价 -----
320
321 def generate_solutions(n: int = 6, method: str = "mixed", seed: int = 42) -> list[dict
    ]:
322     """
323     生成n个分组方案并评价。
324     method: "csp", "greedy", "mixed"

```

```

325
326 """修复：所有方案均施加模拟退火优化，与论文描述一致。"""
327 """
328     solutions = []
329
330     for i in range(n):
331         # 生成初始可行解
332         if method == "csp" or (method == "mixed" and i < 2):
333             sol = generate_csp_solution(seed=seed + i)
334             if sol is None:
335                 sol = generate_greedy_random(seed=seed + i)
336                 base_method = "greedy-fallback"
337             else:
338                 base_method = "csp"
339         else:
340             sol = generate_greedy_random(seed=seed + i)
341             base_method = "greedy"
342
343         # 修复：对所有方案统一施加 SA 优化
344         sol, sa_info = simulated_annealing(
345             sol, max_iter=50000, T_start=100.0, T_end=1.0, alpha=0.995,
346             seed=seed + i
347         )
348         method_used = f"{base_method}+SA"
349
350         eval_result = evaluate_groups(sol)
351         solutions.append({
352             "id": i + 1,
353             "method": method_used,
354             "groups": sol,
355             "eval": eval_result,
356         })
357
358     return solutions
359
360
361 def format_solution_table(sol: dict) -> str:
362     """格式化一个方案为文本表格。"""
363     lines = [
364         f"方案{sol['id']} {sol['method']} "

```

```

365         f"可行={sol['eval']['feasible']}"
366         f"软约束违反={sol['eval']['soft_violations']}"
367     ]
368     lines.append(
369         f"GDP均衡:均值={sol['eval']['balance']['mean']:.0f}亿"
370         f"CV={sol['eval']['balance']['cv']:.4f}"
371     )
372     lines.append("-" * 60)
373     for i, g in enumerate(sol["groups"]):
374         names = [TEAM_BY_CODE[c]["name"] for c in g]
375         gdp = sum(TEAM_BY_CODE[c]["gdp"] for c in g)
376         lines.append(f"组{i+1:2d}:{' '.join(names):30s}GDP={gdp:5d}亿")
377     return "\n".join(lines)

```

Listing 2: 抽签方案代码 (src/lottery.py)

```

1  """
2  Task2: 抽签方案设计
3
4  方法:
5  - 分层随机抽签: 第一层市级队, 第二层县级队
6  - 县级队抽签优先满足软约束, 并通过合法性验签与必要重抽保证硬约束;
7  - 保留随机回溯作为极端兜底方案。
8
9  修复说明:
10  1. 修正原论文公式"允许的组"把软约束变成硬约束的问题。
11  2. 保留贪心抽签作为对照: 优先从"不含同市其他县级队"的未满足组中随机选;
12     若没有此类组, 则从所有符合硬约束的未满足组中随机选。
13  3. 新增验签重抽版作为默认抽签方式, 避免贪心策略走入死胡同导致
14     个别抽签结果不满足硬约束; 随机回溯仅作为极端兜底。
15  """
16
17  import random
18  import numpy as np
19  from collections import defaultdict
20
21  from .data import (
22      ALL_TEAMS, CITY_CODES, COUNTY_CODES, PARENT_MAP,
23      CITY_CHILDREN, NUM_GROUPS, GROUP_SIZE, TEAM_BY_CODE,
24  )

```

```

25
26
27 # -- 第一层：市级队抽签 -----
28
29 def draw_municipal_order(seed: int = None) -> list[str]:
30     """市级队随机排序，返回随机顺序的市级队列表。"""
31     rng = random.Random(seed)
32     order = CITY_CODES[:]
33     rng.shuffle(order)
34     return order
35
36
37 # -- 第二层：县级队抽签（优先软约束版） -----
38
39 def draw_county_assignment_greedy(
40     city_groups: dict[str, int],
41     seed: int = None,
42 ) -> dict[str, int]:
43     """
44     第二层（优先软约束版）：县级队贪心随机分配。
45
46     对于每个市级队代管的县：
47     优先选择满足以下所有条件的组：
48     1. 未满 (< GROUP_SIZE)
49     2. 不含该市市级队（硬约束）
50     3. 不含同市其他已分配的县级队（软约束优先）
51     若没有同时满足以上条件的组，则放宽条件3，
52     从仅满足条件1和2的组中随机选择（允许软约束违反）。
53     """
54     rng = random.Random(seed)
55
56     county_assignment = {}
57     group_count = defaultdict(int)
58
59     # 初始化：市级队已占位
60     for city_code, g_idx in city_groups.items():
61         group_count[g_idx] += 1
62
63     # 逐市分配县级队（随机顺序增加多样性）
64     city_order = CITY_CODES[:]

```

```

65     rng.shuffle(city_order)
66
67     for city_code in city_order:
68         city_group = city_groups[city_code]          # 该市市级队所在组（硬禁止）
69         children = CITY_CHILDREN[city_code][:]
70         rng.shuffle(children)
71
72         # 记录同市已分配的县级队所在的组（用于软约束优先）
73         used_by_same_city = set()
74
75         for child_code in children:
76             # ---- 第一优先级：满足硬约束 + 软约束 ----
77             ideal_groups = []
78             for g_idx in range(NUM_GROUPS):
79                 if (
80                     g_idx != city_group                # 硬约束：不与本市市级队同组
81                     and group_count[g_idx] < GROUP_SIZE # 未满
82                     and g_idx not in used_by_same_city # 软约束优先：不与同市其他
83                                                         县同组
84                 ):
85                     ideal_groups.append(g_idx)
86
87             if ideal_groups:
88                 chosen = rng.choice(ideal_groups)
89             else:
90                 # ---- 第二优先级：放松软约束，仅满足硬约束 ----
91                 fallback_groups = []
92                 for g_idx in range(NUM_GROUPS):
93                     if (
94                         g_idx != city_group
95                         and group_count[g_idx] < GROUP_SIZE
96                     ):
97                         fallback_groups.append(g_idx)
98
99                 if fallback_groups:
100                     chosen = rng.choice(fallback_groups)
101                 else:
102                     # 理论上不应发生（16组×4容量=64队，完全够放）
103                     chosen = rng.randint(0, NUM_GROUPS - 1)

```

```

104         county_assignment[child_code] = chosen
105         group_count[chosen] += 1
106         used_by_same_city.add(chosen) # 记录该组已被本市级队的一个县占用
107
108     return county_assignment
109
110
111 # -- 第二层：随机回溯版（默认，保证硬约束）-----
112
113 def draw_county_assignment_backtracking(
114     city_groups: dict[str, int],
115     seed: int = None,
116 ) -> dict[str, int]:
117     """
118     第二层（随机回溯版）：县级队随机抽签并保证硬约束。
119
120     做法：
121     先尝试把"同市县级队不同组"也作为约束求解；
122     若极端情况下无解，再放宽软约束，但仍保证每组容量和市-县不同组。
123
124     这样单次抽签不会出现硬约束不可行结果，同时保留随机性。
125     """
126     rng = random.Random(seed)
127
128     def solve(enforce_soft: bool) -> dict[str, int] | None:
129         group_capacity = [GROUP_SIZE for _ in range(NUM_GROUPS)]
130         for city_g in city_groups.values():
131             group_capacity[city_g] -= 1
132
133         assignment: dict[str, int] = {}
134         used_by_same_city = {city_code: set() for city_code in CITY_CODES}
135
136         def candidates_for(county_code: str) -> list[int]:
137             parent = PARENT_MAP[county_code]
138             forbidden_group = city_groups[parent]
139             candidates = [
140                 g_idx
141                 for g_idx in range(NUM_GROUPS)
142                 if g_idx != forbidden_group and group_capacity[g_idx] > 0
143             ]

```

```

144     if enforce_soft:
145         candidates = [
146             g_idx for g_idx in candidates
147             if g_idx not in used_by_same_city[parent]
148         ]
149         rng.shuffle(candidates)
150         return candidates
151
152     rng.shuffle(candidates)
153     candidates.sort(
154         key=lambda g_idx: g_idx in used_by_same_city[parent]
155     )
156     return candidates
157
158 def backtrack(remaining: list[str]) -> bool:
159     if not remaining:
160         return True
161
162     scored = []
163     for county_code in remaining:
164         cands = candidates_for(county_code)
165         if not cands:
166             return False
167         scored.append((len(cands), rng.random(), county_code, cands))
168
169     _, _, county_code, candidates = min(scored)
170     parent = PARENT_MAP[county_code]
171     next_remaining = [c for c in remaining if c != county_code]
172
173     for g_idx in candidates:
174         assignment[county_code] = g_idx
175         group_capacity[g_idx] -= 1
176         was_used = g_idx in used_by_same_city[parent]
177         used_by_same_city[parent].add(g_idx)
178
179         if backtrack(next_remaining):
180             return True
181
182     group_capacity[g_idx] += 1
183     del assignment[county_code]

```

```

184         if not was_used:
185             used_by_same_city[parent].remove(g_idx)
186
187         return False
188
189     remaining_counties = COUNTY_CODES[:]
190     rng.shuffle(remaining_counties)
191     if backtrack(remaining_counties):
192         return assignment
193     return None
194
195     strict_solution = solve(enforce_soft=True)
196     if strict_solution is not None:
197         return strict_solution
198
199     relaxed_solution = solve(enforce_soft=False)
200     if relaxed_solution is None:
201         raise RuntimeError("随机回溯抽签无法找到满足硬约束的县级队分配")
202     return relaxed_solution
203
204
205 # -- 第二层：验签重抽版（默认，速度快且保证硬约束）-----
206
207 def draw_county_assignment_retry(
208     city_groups: dict[str, int],
209     seed: int = None,
210     max_attempts: int = 100,
211 ) -> dict[str, int]:
212     """
213     第二层（验签重抽版）：用贪心随机抽签产生候选，再验签。
214
215     若候选结果违反硬约束，则更换随机种子重抽；若多次重抽仍未成功，
216     再调用随机回溯兜底。由于贪心候选的硬约束成功率较高，实际运行很快。
217     """
218     from .grouping import check_hard_constraints
219
220     rng = random.Random(seed)
221     for _ in range(max_attempts):
222         attempt_seed = rng.randint(0, 2**31 - 1)
223         county_groups = draw_county_assignment_greedy(

```

```

224         city_groups, seed=attempt_seed
225     )
226     groups = lottery_to_groups(city_groups, county_groups)
227     ok, _ = check_hard_constraints(groups)
228     if ok:
229         return county_groups
230
231     return draw_county_assignment_backtracking(city_groups, seed)
232
233
234 # -- 第二层备用: CP-SAT 精确版 -----
235
236 def draw_county_assignment_csp(
237     city_groups: dict[str, int],
238     seed: int = None,
239 ) -> dict[str, int]:
240     """
241     第二层（精确版）：用CP-SAT求解，确保每组恰好4队。
242     仅用于单次精确抽签展示，不适合蒙特卡洛大规模模拟。
243     """
244     from ortools.sat.python import cp_model
245
246     model = cp_model.CpModel()
247     n = len(COUNTY_CODES)
248     g = NUM_GROUPS
249
250     x = {}
251     for i in range(n):
252         for j in range(g):
253             x[i, j] = model.NewBoolVar(f"x_{i}_{j}")
254
255     # 每县恰好一队
256     for i in range(n):
257         model.Add(sum(x[i, j] for j in range(g)) == 1)
258
259     # 每组最终恰好 4 队（减去已放入的市级队）
260     group_base = defaultdict(int)
261     for city_g in city_groups.values():
262         group_base[city_g] += 1
263     for j in range(g):

```

```

264         model.Add(sum(x[i, j] for i in range(n)) == GROUP_SIZE - group_base[j])
265
266     # 硬约束：市级队与该市县级队不同组
267     for i, county_code in enumerate(COUNTY_CODES):
268         parent = PARENT_MAP[county_code]
269         parent_group = city_groups[parent]
270         model.Add(x[i, parent_group] == 0)
271
272     # 软约束：同市县级队尽量不同组（不强制）
273     # 注：CP-SAT 版本不主动优化软约束，仅保证硬约束。
274
275     solver = cp_model.CpSolver()
276     solver.parameters.num_workers = 1
277     if seed is not None:
278         solver.parameters.random_seed = seed
279     status = solver.Solve(model)
280
281     if status not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
282         raise RuntimeError("CP-SAT无法找到可行解")
283
284     county_assignment = {}
285     for i, county_code in enumerate(COUNTY_CODES):
286         for j in range(g):
287             if solver.Value(x[i, j]) == 1:
288                 county_assignment[county_code] = j
289                 break
290     return county_assignment
291
292
293 # -- 完整抽签流程 -----
294
295 def run_lottery(seed: int = None, method: str = "retry") -> tuple[dict, dict]:
296     """
297     执行一次完整抽签。
298
299     返回：
300     city_groups: 市级队->所在组编号
301     county_groups: 县级队->所在组编号
302     """
303     # 第一层：市级队抽签

```

```

304     city_order = draw_municipal_order(seed)
305     city_groups = {}
306     for i, city_code in enumerate(city_order):
307         city_groups[city_code] = i % NUM_GROUPS
308
309     # 第二层：县级队抽签
310     if method == "csp":
311         county_groups = draw_county_assignment_csp(city_groups, seed)
312     elif method == "backtracking":
313         county_groups = draw_county_assignment_backtracking(city_groups, seed)
314     elif method == "greedy":
315         county_groups = draw_county_assignment_greedy(city_groups, seed)
316     else:
317         county_groups = draw_county_assignment_retry(city_groups, seed)
318
319     return city_groups, county_groups
320
321
322 def lottery_to_groups(
323     city_groups: dict[str, int],
324     county_groups: dict[str, int],
325 ) -> list[list[str]]:
326     """将抽签结果转化为分组列表格式。"""
327     groups = [[] for _ in range(NUM_GROUPS)]
328     for code, g_idx in city_groups.items():
329         groups[g_idx].append(code)
330     for code, g_idx in county_groups.items():
331         groups[g_idx].append(code)
332     return groups
333
334
335 # -- 蒙特卡洛公平性验证 -----
336
337 def simulate_lottery_fairness(
338     n_simulations: int = 10000,
339     seed: int = 42,
340 ) -> dict:
341     """
342     蒙特卡洛模拟大量抽签，统计：
343

```

```

344 1. hard_feasible_rate: 硬约束全部满足的比例
345 2. violation_mean / violation_std: 软约束违反次数的均值与标准差
346 3. violation_distribution: 软约束违反次数的分布
347 4. uniformity: 各队落入各组的概率均匀性
348 """
349 from .grouping import check_hard_constraints, count_soft_violations
350
351 rng = random.Random(seed)
352
353 team_group_counts = defaultdict(lambda: np.zeros(NUM_GROUPS, dtype=int))
354 violation_counts = []
355 feasible_count = 0
356 violation_dist = defaultdict(int)
357
358 for sim in range(n_simulations):
359     sim_seed = rng.randint(0, 2**31 - 1)
360     city_groups, county_groups = run_lottery(seed=sim_seed, method="retry")
361     groups = lottery_to_groups(city_groups, county_groups)
362
363     # 统计各队在各个组出现的次数
364     for g_idx, g in enumerate(groups):
365         for code in g:
366             team_group_counts[code][g_idx] += 1
367
368     # 硬约束检查
369     ok, _ = check_hard_constraints(groups)
370     if ok:
371         feasible_count += 1
372
373     # 软约束统计
374     v = count_soft_violations(groups)
375     violation_counts.append(v["total_violations"])
376     violation_dist[v["total_violations"]] += 1
377
378 n_actual = len(violation_counts)
379
380 # 均匀性检验: 各队在各组的分布是否均匀
381 uniformity = {}
382 for code in team_group_counts:
383     counts = team_group_counts[code]

```

```

384     total = counts.sum()
385     if total == 0:
386         continue
387     expected = total / NUM_GROUPS
388     chi_sq = float(np.sum((counts - expected) ** 2 / max(expected, 1e-6)))
389     uniformity[code] = {
390         "chi_sq": chi_sq,
391         "max_deviation": float(np.max(np.abs(counts - expected))),
392         "probs": (counts / total).tolist(),
393     }
394
395     return {
396         "n_simulations": n_actual,
397         "feasible_rate": feasible_count / max(n_actual, 1),
398         "violation_mean": float(np.mean(violation_counts)) if violation_counts else 0,
399         "violation_std": float(np.std(violation_counts)) if violation_counts else 0,
400         "violation_distribution": dict(violation_dist),
401         "uniformity": uniformity,
402         "team_group_counts": {k: v.tolist() for k, v in team_group_counts.items()},
403     }
404
405
406 # -- 格式化输出 -----
407
408 def format_lottery_result(city_groups: dict, county_groups: dict) -> str:
409     """格式化一次抽签结果为可读文本。"""
410     groups = [[] for _ in range(NUM_GROUPS)]
411     for code, g in city_groups.items():
412         groups[g].append(code)
413     for code, g in county_groups.items():
414         groups[g].append(code)
415
416     lines = ["抽签结果:", "-" * 50]
417     for i, g in enumerate(groups):
418         names = [TEAM_BY_CODE[c]["name"] for c in g]
419         lines.append(f"└─组{i+1:2d}:└─{' , '.join(names)}")
420     return "\n".join(lines)

```

Listing 3: 赛制分析代码 (src/tournament.py)

```

1  """
2  Task4: 赛制建议与分析
3
4  方法:
5  - 基于GDP构建Bradley-Terry实力模型
6  - 蒙特卡洛模拟三种赛制: 当前赛制、简化瑞士轮制、近似双败淘汰制
7  - 用Gini系数衡量夺冠概率分布离散程度
8  """
9
10 import random
11 import numpy as np
12 from collections import defaultdict
13
14 from .data import ALL_TEAMS, TEAM_BY_CODE
15
16
17 # =====
18 # 实力模型
19 # =====
20
21 def build_strength_model(gdp_weight=0.7,
22                         random_weight=0.3,
23                         seed=42) -> dict[str, float]:
24     """
25     基于GDP构建队伍实力分数。
26     对数归一化+随机噪声，映射到[0.3,1.0]。
27     """
28     rng = np.random.RandomState(seed)
29
30     gdps = np.array([t["gdp"] for t in ALL_TEAMS], dtype=float)
31
32     log_gdp = np.log1p(gdps)
33
34     norm_gdp = (
35         (log_gdp - log_gdp.min())
36         / (log_gdp.max() - log_gdp.min())
37     )
38
39     noise = rng.uniform(0, 1, len(ALL_TEAMS))
40

```

```

41     raw = gdp_weight * norm_gdp + random_weight * noise
42
43     strength = (
44         0.3
45         + 0.7 * (raw - raw.min())
46         / (raw.max() - raw.min())
47     )
48
49     return {
50         ALL_TEAMS[i]["code"]: float(strength[i])
51         for i in range(len(ALL_TEAMS))
52     }
53
54
55 def match_prob(s_a: float, s_b: float) -> float:
56     """Bradley-Terry: A胜B的概率。"""
57     return s_a / (s_a + s_b)
58
59
60 def simulate_match(a: str,
61                   b: str,
62                   strength: dict,
63                   rng: random.Random) -> tuple[str, str]:
64     """
65     模拟一场比赛。
66
67     返回:
68     (胜者代号, 败者代号)
69     """
70     if rng.random() < match_prob(strength[a], strength[b]):
71         return a, b
72     return b, a
73
74
75 # =====
76 # 赛制 1: 当前赛制
77 # 16组单循环 + 32强淘汰赛
78 # =====
79
80 def simulate_group_stage(group_codes: list[str],

```

```

81         strength: dict,
82         rng: random.Random):
83     """
84     单循环小组赛。
85
86     返回:
87     ranking: 排名列表
88     points: 积分字典
89     """
90     points = {c: 0 for c in group_codes}
91
92     for i in range(len(group_codes)):
93         for j in range(i + 1, len(group_codes)):
94             a, b = group_codes[i], group_codes[j]
95
96             winner, loser = simulate_match(
97                 a, b, strength, rng
98             )
99
100             points[winner] += 3
101
102     ranking = sorted(
103         group_codes,
104         key=lambda c: (points[c], strength[c]),
105         reverse=True
106     )
107
108     return ranking, points
109
110
111 def simulate_knockout(teams: list[str],
112                      strength: dict,
113                      rng: random.Random) -> str:
114     """
115     单败淘汰赛。
116
117     返回:
118     champion: 冠军代号
119     """
120     current = list(teams)

```

```

121
122     while len(current) > 1:
123
124         next_round = []
125
126         rng.shuffle(current)
127
128         for i in range(0, len(current), 2):
129
130             if i + 1 < len(current):
131
132                 a, b = current[i], current[i + 1]
133
134                 winner, loser = simulate_match(
135                     a, b, strength, rng
136                 )
137
138                 next_round.append(winner)
139
140             else:
141                 # 奇数队轮空
142                 next_round.append(current[i])
143
144             current = next_round
145
146     return current[0]
147
148
149 def simulate_current_format(groups: list[list[str]],
150                             strength: dict,
151                             rng: random.Random):
152
153     """
154     当前赛制:
155     16组单循环+32强淘汰赛
156
157     返回:
158     champion
159     all_points
160     qualifiers
161     """

```

```

161     qualifiers = []
162
163     all_points = {}
164
165     for g in groups:
166
167         ranking, points = simulate_group_stage(
168             g, strength, rng
169         )
170
171         qualifiers.extend(ranking[:2])
172
173         all_points.update(points)
174
175     rng.shuffle(qualifiers)
176
177     champion = simulate_knockout(
178         qualifiers,
179         strength,
180         rng
181     )
182
183     return champion, all_points, qualifiers
184
185
186 # =====
187 # 赛制 2: 简化瑞士轮制
188 # =====
189
190 def simulate_swiss_system(all_codes: list[str],
191                           strength: dict,
192                           n_rounds: int = 6,
193                           rng: random.Random = None):
194     """
195     简化瑞士轮近似模型:
196
197     按当前积分排序后相邻配对
198     不考虑重复对阵规避等标准瑞士轮细节
199     6轮后取前32进入淘汰赛
200     """

```

```

201     if rng is None:
202         rng = random.Random()
203
204     points = {c: 0.0 for c in all_codes}
205
206     for _ in range(n_rounds):
207
208         sorted_teams = sorted(
209             all_codes,
210             key=lambda c: (points[c], strength[c]),
211             reverse=True
212         )
213
214         for i in range(0, len(sorted_teams) - 1, 2):
215
216             a = sorted_teams[i]
217             b = sorted_teams[i + 1]
218
219             winner, loser = simulate_match(
220                 a, b, strength, rng
221             )
222
223             points[winner] += 1.0
224
225     ranking = sorted(
226         all_codes,
227         key=lambda c: (points[c], strength[c]),
228         reverse=True
229     )
230
231     top32 = ranking[:32]
232
233     champion = simulate_knockout(
234         top32,
235         strength,
236         rng
237     )
238
239     return champion, points, top32
240

```

```

241
242 # =====
243 # 赛制 3: 近似双败淘汰制
244 # =====
245
246 def simulate_double_elimination(all_codes: list[str],
247                                strength: dict,
248                                rng: random.Random) -> str:
249     """
250     近似双败淘汰机制:
251
252     说明:
253     - 先进行 3 轮积分预选
254     - 取前 32 名进入淘汰阶段
255     - 输一场进入败者组
256     - 输两场后淘汰
257     - 并非严格标准双败赛程
258     """
259
260     # -----
261     # 预选: 3轮积分赛
262     # -----
263
264     points = {c: 0.0 for c in all_codes}
265
266     for _ in range(3):
267
268         sorted_teams = sorted(
269             all_codes,
270             key=lambda c: (points[c], strength[c]),
271             reverse=True
272         )
273
274         for i in range(0, len(sorted_teams) - 1, 2):
275
276             a = sorted_teams[i]
277             b = sorted_teams[i + 1]
278
279             winner, loser = simulate_match(
280                 a, b, strength, rng

```

```

281         )
282
283         points[winner] += 1.0
284
285     ranking = sorted(
286         all_codes,
287         key=lambda c: (points[c], strength[c]),
288         reverse=True
289     )
290
291     top32 = ranking[:32]
292
293     # -----
294     # 双败阶段
295     # -----
296
297     winners = list(top32)
298
299     losers = []
300
301     rng.shuffle(winners)
302
303     while True:
304
305         # -----
306         # 胜者组
307         # -----
308
309         next_winners = []
310
311         for i in range(0, len(winners), 2):
312
313             if i + 1 < len(winners):
314
315                 a = winners[i]
316                 b = winners[i + 1]
317
318                 win, lose = simulate_match(
319                     a, b, strength, rng
320                 )

```

```

321
322         next_winners.append(win)
323
324         losers.append(lose)
325
326     else:
327         next_winners.append(winners[i])
328
329     winners = next_winners
330
331     # -----
332     # 败者组
333     # -----
334
335     next_losers = []
336
337     for i in range(0, len(losers), 2):
338
339         if i + 1 < len(losers):
340
341             a = losers[i]
342             b = losers[i + 1]
343
344             win, lose = simulate_match(
345                 a, b, strength, rng
346             )
347
348             next_losers.append(win)
349
350         else:
351             next_losers.append(losers[i])
352
353     losers = next_losers
354
355     # -----
356     # 决赛
357     # -----
358
359     if len(winners) == 1 and len(losers) <= 1:
360

```

```

361         if len(losers) == 1:
362
363             a = winners[0]
364             b = losers[0]
365
366             winner, loser = simulate_match(
367                 a, b, strength, rng
368             )
369
370             return winner
371
372         return winners[0]
373
374
375 # =====
376 # 蒙特卡洛模拟
377 # =====
378
379 def monte_carlo_comparison(groups: list[list[str]],
380                             n_simulations: int = 5000,
381                             seed: int = 42):
382     """
383     对比三种赛制下各队夺冠概率。
384
385     返回：
386     probs
387     ginis
388     strength
389     """
390
391     rng = random.Random(seed)
392
393     strength = build_strength_model(seed=seed)
394
395     all_codes = [t["code"] for t in ALL_TEAMS]
396
397     results = {
398         "current": defaultdict(int),
399         "swiss": defaultdict(int),
400         "double_elim": defaultdict(int),

```

```

401     }
402
403     for _ in range(n_simulations):
404
405         # 当前赛制
406         sim_rng = random.Random(
407             rng.randint(0, 2**31 - 1)
408         )
409
410         champ, _, _ = simulate_current_format(
411             groups,
412             strength,
413             sim_rng
414         )
415
416         results["current"][champ] += 1
417
418         # 瑞士轮
419         sim_rng2 = random.Random(
420             rng.randint(0, 2**31 - 1)
421         )
422
423         champ, _, _ = simulate_swiss_system(
424             all_codes,
425             strength,
426             6,
427             sim_rng2
428         )
429
430         results["swiss"][champ] += 1
431
432         # 双败
433         sim_rng3 = random.Random(
434             rng.randint(0, 2**31 - 1)
435         )
436
437         champ = simulate_double_elimination(
438             all_codes,
439             strength,
440             sim_rng3

```

```

441         )
442
443         results["double_elim"][champ] += 1
444
445     probs = {
446         fmt: {
447             code: cnt / n_simulations
448             for code, cnt in counts.items()
449         }
450         for fmt, counts in results.items()
451     }
452
453     ginis = {}
454
455     for fmt in ["current", "swiss", "double_elim"]:
456
457         vals = np.array([
458             probs[fmt].get(t["code"], 0)
459             for t in ALL_TEAMS
460         ])
461
462         ginis[fmt] = _gini(vals)
463
464     return probs, ginis, strength
465
466
467 def analyze_competitiveness(groups, strength, n_simulations=2000, seed=42):
468     """分析竞技水平展示度：弱队平均比赛场数和出线率。"""
469     rng = random.Random(seed)
470
471     sorted_teams = sorted(ALL_TEAMS, key=lambda t: strength[t["code"]])
472     weak_codes = set(t["code"] for t in sorted_teams[:13])
473
474     weak_qualify = defaultdict(int)
475
476     for sim in range(n_simulations):
477         sim_rng = random.Random(rng.randint(0, 2**31 - 1))
478         for g in groups:
479             ranking, _ = simulate_group_stage(g, strength, sim_rng)
480             for code in g:

```

```

481         if code in weak_codes and code in ranking[:2]:
482             weak_qualify[code] += 1
483
484     qualify_rate = {c: weak_qualify[c] / n_simulations for c in weak_codes}
485
486     return {
487         "weak_teams": list(weak_codes),
488         "avg_matches": {c: 3.0 for c in weak_codes}, # 小组赛固定3场
489         "qualify_rate": qualify_rate,
490         "avg_qualify_rate": float(np.mean(list(qualify_rate.values()))),
491     }
492
493
494 # =====
495 # Gini 系数
496 # =====
497
498 def _gini(x: np.ndarray) -> float:
499     """
500     计算Gini系数。
501
502     Gini越低,
503     说明夺冠概率分布越均衡。
504     """
505     x = np.sort(x)
506
507     n = len(x)
508
509     if n == 0 or np.sum(x) == 0:
510         return 0.0
511
512     g = (
513         2 * np.sum(np.arange(1, n + 1) * x)
514         - (n + 1) * np.sum(x)
515     ) / (n * np.sum(x))
516
517     return float(max(g, 0.0))
518
519
520 def _rankdata_average(x: np.ndarray) -> np.ndarray:

```

```

521     """返回平均秩，用于处理并列值的_Spearman_相关。"""
522     x = np.asarray(x, dtype=float)
523     n = len(x)
524     order = np.argsort(x, kind="mergesort")
525     ranks = np.empty(n, dtype=float)
526
527     i = 0
528     while i < n:
529         j = i
530         while j + 1 < n and x[order[j + 1]] == x[order[i]]:
531             j += 1
532         avg_rank = (i + j) / 2.0 + 1.0
533         ranks[order[i:j + 1]] = avg_rank
534         i = j + 1
535
536     return ranks
537
538
539 def _spearman_corr(x: np.ndarray, y: np.ndarray) -> float:
540     """Spearman_秩相关系数。"""
541     rx = _rankdata_average(x)
542     ry = _rankdata_average(y)
543     rx = rx - np.mean(rx)
544     ry = ry - np.mean(ry)
545     denom = np.sqrt(np.sum(rx ** 2) * np.sum(ry ** 2))
546     if denom == 0:
547         return 0.0
548     return float(np.sum(rx * ry) / denom)
549
550
551 def champion_alignment_metrics(probs: dict,
552                                strength: dict,
553                                top_n: int = 16) -> dict:
554     """
555     分析赛制是否保留竞技区分度。
556
557     指标:
558     -_Spearman: 实力分数与夺冠概率的秩相关，越高说明强队优势越能反映在结果中。
559     -_top_strength_share: 实力前_top_n_队的总夺冠概率占比。
560     """

```

```

561 codes = [t["code"] for t in ALL_TEAMS]
562 strength_vals = np.array([strength.get(c, 0.0) for c in codes])
563 top_strength = set(
564     sorted(codes, key=lambda c: strength.get(c, 0.0), reverse=True)[:top_n]
565 )
566
567 metrics = {}
568 for fmt in ["current", "swiss", "double_elim"]:
569     prob_vals = np.array([probs[fmt].get(c, 0.0) for c in codes])
570     top_prob = set(
571         sorted(codes, key=lambda c: probs[fmt].get(c, 0.0), reverse=True)[:top_n]
572     )
573     metrics[fmt] = {
574         "spearman": _spearman_corr(strength_vals, prob_vals),
575         "top_strength_share": float(sum(probs[fmt].get(c, 0.0) for c in
576             top_strength)),
577         "top_overlap": len(top_strength & top_prob) / top_n,
578     }
579
580 return metrics
581
582 # =====
583 # 格式化输出
584 # =====
585
586 def format_tournament_comparison(probs: dict,
587     strength: dict) -> str:
588     """
589     格式化赛制对比结果。
590     """
591
592     lines = [
593         "赛制对比分析",
594         "=" * 70
595     ]
596
597     top_teams = sorted(
598         ALL_TEAMS,
599         key=lambda t: strength.get(t["code"], 0),

```

```

600         reverse=True
601    )[:15]
602
603     header = (
604         f"{'队伍':>8s}__"
605         f"{'实力':>6s}__"
606         f"{'当前赛制':>10s}__"
607         f"{'瑞士轮':>10s}__"
608         f"{'近似双败':>10s}"
609     )
610
611     lines.append(header)
612
613     lines.append("-" * 70)
614
615     for t in top_teams:
616
617         code = t["code"]
618
619         name = t["name"]
620
621         s = strength.get(code, 0)
622
623         p1 = probs["current"].get(code, 0)
624
625         p2 = probs["swiss"].get(code, 0)
626
627         p3 = probs["double_elim"].get(code, 0)
628
629         lines.append(
630             f"{name:>8s}__"
631             f"{s:.4f}____"
632             f"{p1:10.4f}____"
633             f"{p2:10.4f}____"
634             f"{p3:10.4f}"
635         )
636
637     lines.append("\n")
638
639     for fmt, label in [

```

```

640     ("current", "当前赛制"),
641     ("swiss", "瑞士轮"),
642     ("double_elim", "近似双败"),
643 ]:
644
645     vals = np.array([
646         probs[fmt].get(t["code"], 0)
647         for t in ALL_TEAMS
648     ])
649
650     gini = _gini(vals)
651
652     lines.append(
653         f"{label} Gini系数: {gini:.4f}"
654     )
655
656     alignment = champion_alignment_metrics(probs, strength)
657
658     lines.append("\n实力-结果一致性分析")
659     lines.append("-" * 70)
660
661     for fmt, label in [
662         ("current", "当前赛制"),
663         ("swiss", "瑞士轮"),
664         ("double_elim", "近似双败"),
665     ]:
666         m = alignment[fmt]
667         lines.append(
668             f"{label} Spearman相关: {m['spearman']:.4f} "
669             f"实力前16队夺冠概率占比: {m['top_strength_share']:.4f} "
670             f"前16重合率: {m['top_overlap']:.4f}"
671         )
672
673     return "\n".join(lines)

```